

19(a) Signal Handler Registration

What it does: Registers a signal handler, handles SIGINT, and keeps the program stable when an interrupt occurs.

```
sri_dhruthi@Dhruthi: ~/Skill x + v
GNU nano 8.7.1 skill19a.c *
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void handle_sigint(int sig)
{
    printf("\nSIGINT received. Signal handler executed.\n");
}

int main()
{
    signal(SIGINT, handle_sigint);

    printf("SIGINT handler registered.\n");
    printf("PID: %d\n", getpid());
    printf("Press Ctrl+C to test the signal.\n");

    sleep(5);

    printf("Program completed safely.\n");

    return 0;
}
```

```
sri_dhruthi@Dhruthi:~$ cd Skill
sri_dhruthi@Dhruthi:~/Skill$ nano skill19a.c
sri_dhruthi@Dhruthi:~/Skill$ gcc skill19a.c -o skill19a
sri_dhruthi@Dhruthi:~/Skill$ ./skill19a
SIGINT handler registered.
PID: 501
Press Ctrl+C to test the signal.
Program completed safely.
sri_dhruthi@Dhruthi:~/Skill$ |
```

19(b) Capture SIGINT and Forward Signals

What it does: Protects the shell process, forwards SIGINT to the foreground job, and updates the job state after termination.

```

GNU nano 8.7.1 skill19b.c *
#include <stdio.h>
#include <signal.h>
#include <unistd.h>
#include <sys/wait.h>

pid_t foreground_pid;

void handle_sigint(int sig)
{
    if (foreground_pid > 0)
    {
        printf("\nShell: SIGINT received.\n");
        printf("Shell: Forwarding SIGINT to foreground job.\n");

        kill(foreground_pid, SIGINT);
    }
}

int main()
{
    int status;

    foreground_pid = fork();

    if (foreground_pid < 0)
    {
        perror("fork");
        return 1;
    }

    if (foreground_pid == 0)
    {
        signal(SIGINT, SIG_DFL);

        printf("Foreground job started. PID = %d\n", getpid());

        sleep(10);

```

```

        printf("Foreground job completed.\n");

        return 0;
    }

    signal(SIGINT, handle_sigint);

    printf("Shell started.\n");
    printf("Foreground job PID = %d\n", foreground_pid);

    sleep(2);

    printf("Sending SIGINT to foreground job...\n");

    kill(getpid(), SIGINT);

    waitpid(foreground_pid, &status, 0);

    if (WIFSIGNALED(status))
    {
        printf("Foreground job terminated by signal.\n");
    }

    printf("Shell continues running.\n");

    return 0;
}

```

```

Program completed successfully.
sri_dhruthi@Dhruthi:~/Skill$ nano skill19b.c
sri_dhruthi@Dhruthi:~/Skill$ gcc skill19b.c -o skill19b
sri_dhruthi@Dhruthi:~/Skill$ ./skill19b
Shell started.
Foreground job PID = 548
Foreground job started. PID = 548
Sending SIGINT to foreground job...

Shell: SIGINT received.
Shell: Forwarding SIGINT to foreground job.
Foreground job terminated by signal.
Shell continues running.
sri_dhruthi@Dhruthi:~/Skill$

```